CLASS ASSIGNMENT – 6
ID : 12341550
FIFO

STEP 1
FILE : proc.h

```
 #define MAX_PAGES 20
…
under struct proc
…
 int pages[MAX_PAGES];
 int page_count;
```

STEP 2
FILE : vm.c

modified allocum() :

```
int allocuvm(pde_t *pgdir, uint oldsz, uint newsz)
{
  char *mem;
  uint a;
  if (newsz >= KERNBASE)
    return 0;
  if (newsz < oldsz)
    return oldsz;
  a = PGROUNDUP(oldsz);
  for (; a < newsz; a += PGSIZE)
  {
    mem = kalloc();
    if (mem == 0)
    {
      cprintf("allocuvm out of memory\n");
```

```
      deallocuvm(pgdir, newsz, oldsz);
      return 0;
    }
    memset(mem, 0, PGSIZE);
    if (mappages(pgdir, (char *)a, PGSIZE, V2P(mem),
PTE_W | PTE_U) < 0)
    {
      kfree(mem);
      deallocuvm(pgdir, newsz, oldsz);
      return 0;
    }
    struct proc *cur = myproc();
    if (cur->page_count < MAX_PAGES)
    {
      cur->pages[cur->page_count++] = (int)a;
      cprintf("Allocated page %d at virtual address 0x%x\n",
cur->page_count - 1, a);
    }
    else
    {
      int evict = cur->pages[0];
      mem = (char *)P2V(PTE_ADDR(*walkpgdir(pgdir,
(void *)evict, 0)));
      kfree(mem);
      for (int i = 1; i < MAX_PAGES; i++)
        cur->pages[i - 1] = cur->pages[i];
      cur->pages[MAX_PAGES - 1] = (int)a;
      cprintf("Evicted page at 0x%x, allocated new page at 0x
%x\n", evict,
              a);
    }
  }
  return newsz;
}
```

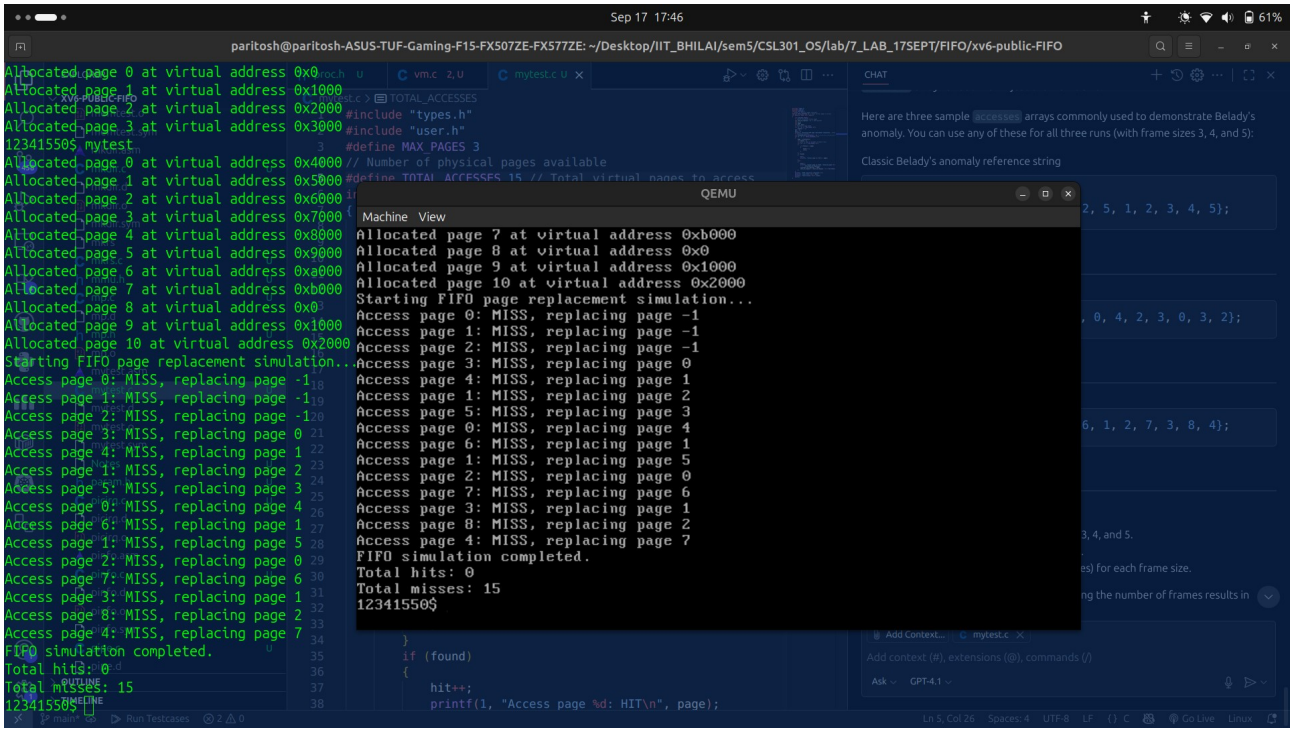
STEP 3
FILE : mytest.c

```
#include "types.h"
#include "user.h"
#define MAX_PAGES 5
// Number of physical pages available
#define TOTAL_ACCESSES 15 // Total virtual pages to
access
int main(int argc, char *argv[])
{
   int fifo[MAX_PAGES];
   // Stores allocated virtual pages
   int next_to_replace = 0; // FIFO pointer
   int i, j;
   int page;
   int hit, miss;
   // Initialize page table
   for (i = 0; i < MAX_PAGES; i++)
      fifo[i] = -1;
   hit = 0;
   miss = 0;
   printf(1, "Starting FIFO page replacement simulation...\
n");
   // Simulated accesses
   int accesses[TOTAL_ACCESSES] = {0, 1, 2, 3, 4, 1, 5, 0,
6, 1, 2, 7, 3, 8, 4};
   for (i = 0; i < TOTAL_ACCESSES; i++)
   {
      page = accesses[i];
      int found = 0;
      // Check if page is already in memory (hit)
      for (j = 0; j < MAX_PAGES; j++)
      {
```

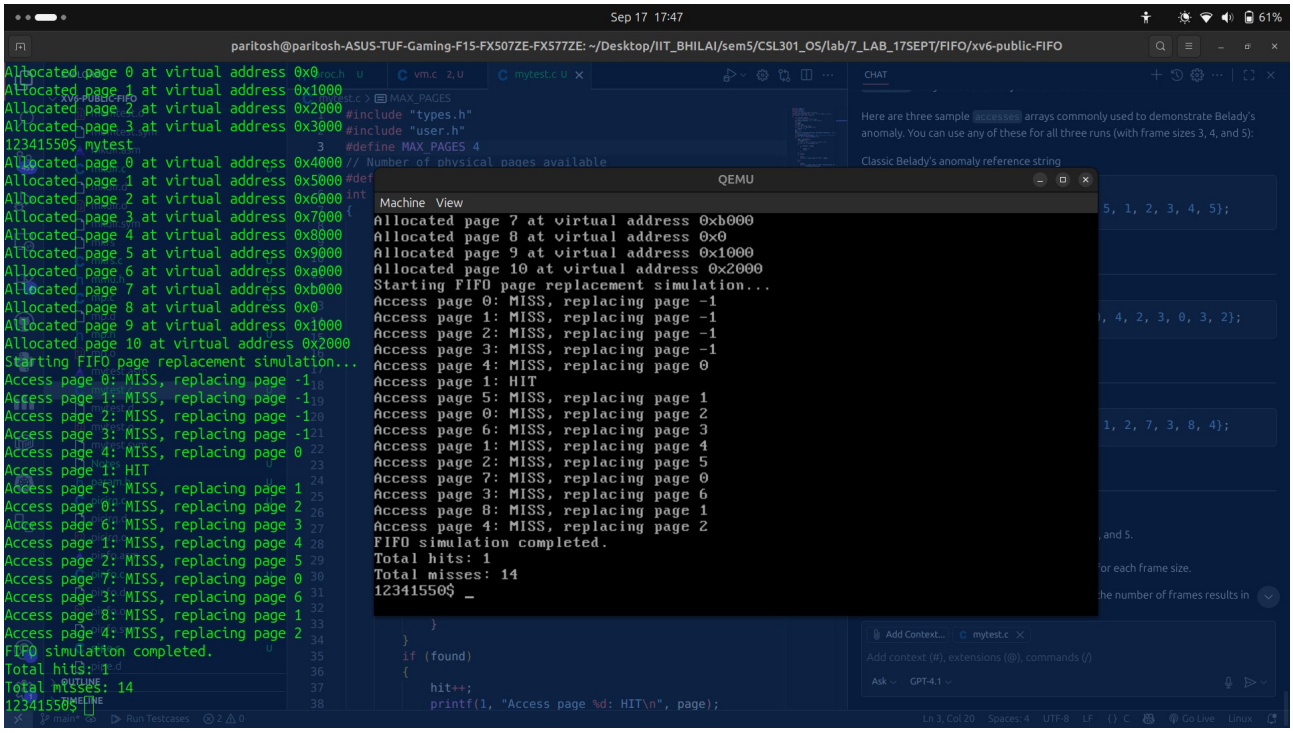
```
            if (fifo[j] == page)
            {
                found = 1;
                break;
            }
        }
        if (found)
        {
            hit++;
            printf(1, "Access page %d: HIT\n", page);
        }
        else
        {
            miss++;
            printf(1, "Access page %d: MISS, replacing page %d\
n", page,
                    fifo[next_to_replace]);
            fifo[next_to_replace] = page;
            next_to_replace = (next_to_replace + 1) %
MAX_PAGES;
        }
    }
    printf(1, "FIFO simulation completed.\n");
    printf(1, "Total hits: %d\n", hit);
    printf(1, "Total misses: %d\n", miss);
    exit();
}
```

OUTPUT :

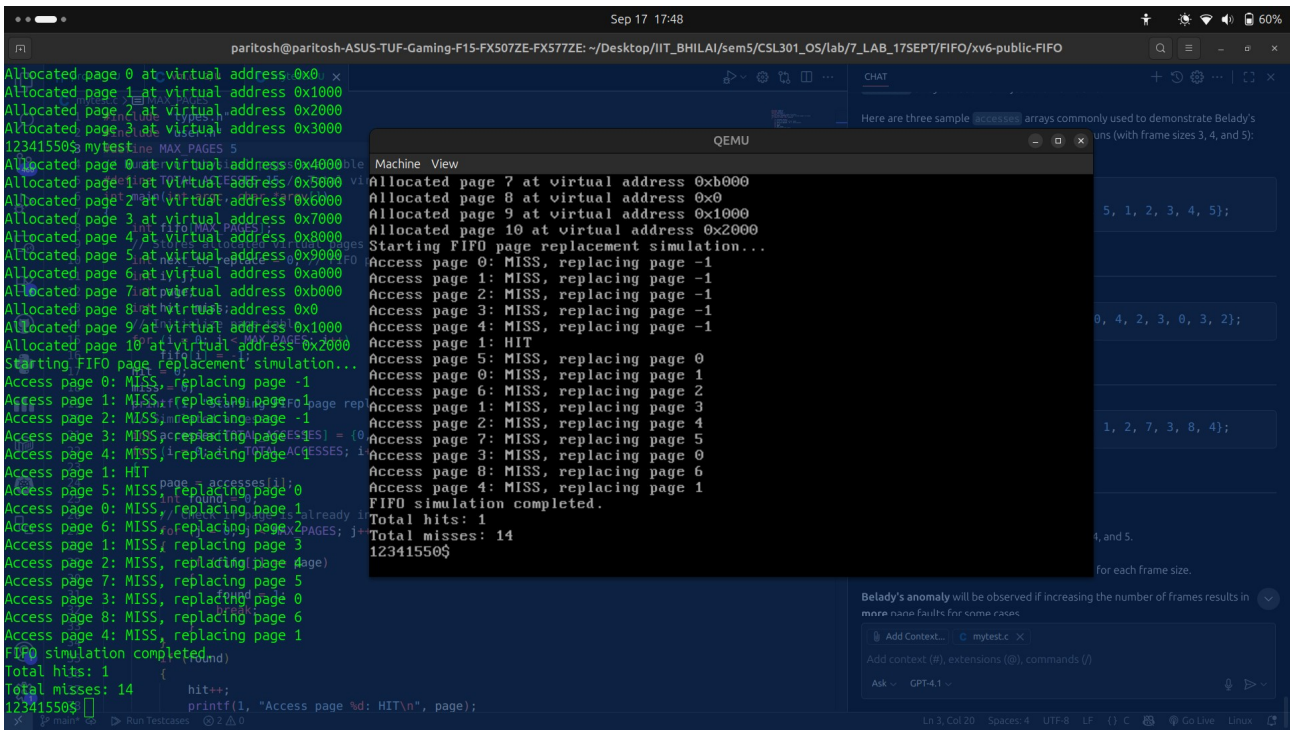
with size 3



with size 4



with size 5


Sep 17 17:48
paritosh@paritosh-ASUS-TUF-Gaming-F15-FX507ZE-FX577ZE: ~/Desktop/IIT_BHILAI/sem5/CSL301_OS/lab/7_LAB_17SEPT/FIFO/xv6-public-FIFO
Allocated page 0 at virtual address 0x0
Allocated page 1 at virtual address 0x1000
Allocated page 2 at virtual address 0x2000
Allocated page 3 at virtual address 0x3000
12341550$ mytest
Allocated page 0 at virtual address 0x4000
Allocated page 1 at virtual address 0x5000
Allocated page 2 at virtual address 0x6000
Allocated page 3 at virtual address 0x7000
Allocated page 4 at virtual address 0x8000
Allocated page 5 at virtual address 0x9000
Allocated page 6 at virtual address 0xa000
Allocated page 7 at virtual address 0xb000
Allocated page 8 at virtual address 0x0
Allocated page 9 at virtual address 0x1000
Allocated page 10 at virtual address 0x2000
Starting FIFO page replacement simulation...
Access page 0: MISS, replacing page -1
Access page 1: MISS, replacing page -1
Access page 2: MISS, replacing page -1
Access page 3: MISS, replacing page -1
Access page 4: MISS, replacing page -1
Access page 1: HIT
Access page 5: MISS, replacing page 0
Access page 0: MISS, replacing page 1
Access page 6: MISS, replacing page 2
Access page 1: MISS, replacing page 3
Access page 2: MISS, replacing page 4
Access page 7: MISS, replacing page 5
Access page 3: MISS, replacing page 0
Access page 8: MISS, replacing page 6
Access page 4: MISS, replacing page 1
FIFO simulation completed.
Total hits: 1
Total misses: 14
12341550$
QEMU
Machine View
Allocated page 7 at virtual address 0xb000
Allocated page 8 at virtual address 0x0
Allocated page 9 at virtual address 0x1000
Allocated page 10 at virtual address 0x2000
Starting FIFO page replacement simulation...
Access page 0: MISS, replacing page -1
Access page 1: MISS, replacing page -1
Access page 6: MISS, replacing page 2
Access page 2: MISS, replacing page -1
Access page 3: MISS, replacing page -1
Access page 4: MISS, replacing page -1
Access page 1: HIT
Access page 5: MISS, replacing page 0
Access page 0: MISS, replacing page 1
Access page 6: MISS, replacing page 2
Access page 1: MISS, replacing page 3
Access page 2: MISS, replacing page 4
Access page 7: MISS, replacing page 5
Access page 3: MISS, replacing page 0
Access page 8: MISS, replacing page 6
Access page 4: MISS, replacing page 1
FIFO simulation completed.
Total hits: 1
Total misses: 14
12341550$
CHAT
Here are three sample accesses arrays commonly used to demonstrate Belady's
5, 1, 2, 3, 4, 5};
0, 4, 2, 3, 0, 3, 2};
1, 2, 7, 3, 8, 4};
4, and 5.
for each frame size.
Belady's anomaly will be observed if increasing the number of frames results in
more page faults for some cases.
Add Context...
mytest.c
Add context (#), extensions (@), commands (/)
Ask
GPT-4.1